

Course-End Project

AI-Powered LinkedIn Content Automation with n8n and AutoGen Microservice

Objective: To design, build, and deploy an AI-powered workflow that automates LinkedIn content creation for a fintech organization — combining a FastAPI-based AutoGen-style microservice (for idea generation, drafting, and hashtags) with an n8n workflow (for scheduling, approval, and publishing).

Tools required: Ubuntu VM, Python 3.10+, FastAPI, Uvicorn, Docker, n8n, Slack (optional), LinkedIn Developer account (optional)

Prerequisites: Basic Python knowledge; familiarity with n8n and MCP-style workflows (Lesson 02, Demo 01 and Demo 02); an Ubuntu VM with internet access

Steps to be followed

- Set up the environment (Node.js, Python, n8n, FastAPI)
- Build the AutoGen-style microservice (/linkedin endpoint)
- Test the microservice locally with curl
- Create the n8n workflow and add the Schedule Trigger
- Add and configure the Brand Config node
- Connect the microservice using an HTTP Request node
- Compose the final post using a Set node
- Build the Approval Gate (confidence score + dry-run check)
- Integrate Slack for dry-run review
- Prepare the LinkedIn node for live publishing
- Add logging and error handling
- Test the complete workflow end-to-end
- Document your work and submit deliverables

Overview

This project challenges you to design and implement an AI-powered workflow automation system that streamlines LinkedIn content creation and posting for a fintech organization. The solution combines an AutoGen-inspired microservice with the n8n workflow orchestrator. The workflow demonstrates intelligent orchestration across multiple AI-driven components, including idea generation, drafting, and hashtag generation, before applying guardrails for approval and publishing. The project highlights how product leaders can deploy automation to achieve efficiency, brand consistency, and scale in content strategy.

Situation

FinEdge Mumbai, a fintech company with over five hundred employees, is facing significant challenges in its LinkedIn strategy:

- The content team spends 15+ hours per week manually drafting posts.
- Inconsistent posting has reduced engagement by 45%.
- Manual content creation is too slow, causing posts to miss trending topics.

- Executives lack a unified voice, weakening brand authority.

Despite having rich domain expertise, the company struggles to maintain an authoritative LinkedIn presence. To overcome this, the product team decides to implement an AutoGen-style multi-agent workflow that generates consistent, on-brand LinkedIn posts with minimal human effort, while retaining oversight through approval controls.

Tasks

- Build a multi-agent microservice that returns multiple post ideas, a draft post with a confidence score, and relevant hashtags.
- Deploy and test the microservice on your VM, ensuring it accepts brand and context information via an API and returns structured JSON output.
- Configure an n8n workflow with nodes for scheduling, brand configuration, microservice invocation, composing the final post, an approval gate, and routing to Slack or LinkedIn.
- Include mechanisms for logging and error handling to ensure transparency and traceability.

Instructions — Detailed Walkthrough

Follow the steps below in order. Each step builds on the previous one, so complete and test each one before moving to the next.

Step 1: Set up the environment

1. Open the lab and launch the Konsole terminal. Use Konsole (not the Cursor AI terminal) for all the Python and microservice commands in Steps 1–3.
2. In Konsole, confirm Python is installed:

```
python3 --version
```

3. Create a project folder for the microservice and move into it:

```
mkdir linkedin-automation && cd linkedin-automation python3 -m venv venv source venv/bin/activate
```

4. Install the Python packages you will need:

```
pip install fastapi uvicorn python-dotenv pydantic
```

Note: Keep this Konsole window open — you will use it again in Step 2 and Step 3 to build and run the microservice.

5. Node.js and Docker are needed later for n8n, not for the microservice. Open the Cursor AI terminal (as in Demo 02) and install them there:

```
curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash - sudo apt-get install -y nodejs  
curl -fsSL https://get.docker.com -o get-docker.sh sh get-docker.sh docker --version
```

Step 2: Build the AutoGen-style microservice

1. Inside the project folder, create two files: main.py and .env.example.
2. In .env.example, add placeholders for any keys your microservice needs (for example, an LLM API key):

```
LLM_API_KEY=Your_api_key_here CONFIDENCE_THRESHOLD=0.7
```

3. In main.py, set up a FastAPI app with a single POST endpoint named /linkedin.
4. The /linkedin endpoint should accept a JSON request body containing brand configuration and context, for example: company name, tone of voice, target audience, and a topic or trending theme.
5. Inside the endpoint, generate three things and return them as structured JSON:
 - ideas — a short list of possible post angles related to the given topic
 - draft — one finished post, written in the brand's tone, together with a confidence score between 0 and 1
 - hashtags — a small set of relevant hashtags for the post
6. Wrap the core logic in a try/except block so the endpoint returns a clear error message (instead of crashing) if something goes wrong.
7. Save the file.

Note: If you are unsure how to structure the AutoGen-style logic, review the Lesson 02 material on multi-agent design before writing main.py.

Step 3: Test the microservice locally

1. In your Konsole terminal, start the microservice with Uvicorn:

```
uvicorn main:app --reload --port 8000
```

2. Open a second Konsole terminal window or tab (keep the server running in the first one — do not close it).
3. Send a test request using curl:

```
curl -X POST http://127.0.0.1:8000/linkedin \ -H "Content-Type: application/json" \  
-d '{"company":"FinEdge","tone":"authoritative","topic":"digital payments trends"}'
```

4. Confirm the response is valid JSON and contains ideas, draft, confidence, and hashtags.
5. Try an incorrect request (for example, missing a required field) and confirm the microservice returns a clear error instead of crashing.

Note: Keep a screenshot of a successful curl response — you will need it for your submission.

Step 4: Create the n8n workflow and add the Schedule Trigger

1. In the Cursor AI terminal, start n8n using Docker, the same way you did in Demo 02:

```
sudo docker run -it --rm --name n8n -p 5678:5678 -v ~/.n8n:/home/node/.n8n n8nio/n8n
```

2. Open a browser and go to <http://localhost:5678>, then sign in.
3. Click Create Workflow in the top-right corner.
4. Click Add First Step, search for Schedule Trigger, and select it.
5. Set a suitable cadence for LinkedIn posting (for example, once per weekday), then click Back to canvas.

Step 5: Add and configure the Brand Config node

1. Click the + button after the Schedule Trigger.
2. Search for the Set node (or Edit Fields) and add it to the canvas.

3. Rename it Brand Config.
4. Add fields for company name, tone of voice, target audience, and topic — these values will be sent to your microservice.
5. Click Back to canvas, then Save the workflow.

Step 6: Connect the microservice with an HTTP Request node

1. Click the + button after the Brand Config node.
2. Search for HTTP Request and add it to the canvas. Rename it AutoGen Microservice.
3. Set the Method to POST and the URL to your microservice endpoint:

```
http://127.0.0.1:8000/linkedin
```

4. Under Body, choose JSON and map each field (company, tone, topic) to the corresponding value from the Brand Config node.
5. Click Execute Node to test the connection and confirm the response contains ideas, draft, confidence, and hashtags.
6. Click Back to canvas, then Save the workflow.

Step 7: Compose the final post with a Set node

1. Click the + button after the AutoGen Microservice node.
2. Add another Set node and rename it Compose Final.
3. Combine the draft text and hashtags from the microservice response into a single field named final_post — no JavaScript is required for this.
4. Click Execute Node to confirm final_post looks correct, then click Back to canvas.

Step 8: Build the Approval Gate

1. Click the + button after the Compose Final node.
2. Search for the IF node and add it to the canvas. Rename it Approval Gate.
3. Add a condition that checks whether the confidence score from the microservice is greater than or equal to your chosen threshold (for example, 0.7).
4. Add a second condition that checks the value of a dry_run flag (true or false).
5. Configure the two output branches:
 - If dry_run is true, or the confidence score is below the threshold — route to Slack for human review
 - If dry_run is false and the confidence score meets the threshold — route to LinkedIn for direct publishing
6. Click Back to canvas, then Save the workflow.

Step 9: Integrate Slack for dry-run testing

1. On the Slack branch of the Approval Gate, click the + button and search for the Slack node.
2. Choose Send Message, and set up credentials using either an Incoming Webhook or OAuth (OAuth is optional).
3. Map the final_post field into the Slack message body so reviewers can see the draft post.

4. Click Execute Node to send a test message and confirm it appears in your chosen Slack channel.
5. Click Back to canvas, then Save the workflow.

Step 10: Prepare the LinkedIn node for live publishing

1. On the LinkedIn branch of the Approval Gate, click the + button and search for the LinkedIn node.
2. Set up LinkedIn credentials using the LinkedIn Developer app you created for this project.
3. Map the final_post field into the LinkedIn post content.
4. You do not need to publish live posts during development — it is enough to configure the node correctly and confirm the mapping with Execute Node using a test/dummy value.

Note: Keep `dry_run` set to `true` while you are testing, so posts are always routed to Slack and never published live by accident.

Step 11: Add logging and error handling

1. Add a node after the Approval Gate branches (for example, a Google Sheets, Airtable, or a simple database node) to log each run.
2. For every run, log at minimum: timestamp, draft text, confidence score, and which branch was taken (Slack or LinkedIn).
3. Add error-handling by enabling Continue On Fail (or an Error Trigger node) on the AutoGen Microservice node, so a failed API call is logged instead of stopping the whole workflow silently.
4. Click Save.

Step 12: Test the complete workflow end-to-end

1. Manually trigger the workflow from the start (Schedule Trigger or Execute Workflow button).
2. Confirm data flows correctly through every node: Brand Config → AutoGen Microservice → Compose Final → Approval Gate → Slack or LinkedIn → Logging.
3. Test both branches of the Approval Gate by changing the `dry_run` flag and the confidence threshold.
4. Deliberately break something small (for example, stop the microservice) to confirm your error handling works, then restart the microservice and re-test.
5. Take screenshots of each successful run for your submission.

Step 13: Document your work and submit

1. Write a short README describing your architecture, setup steps, and how to run the project.
2. Summarize your design decisions, challenges you faced, and any trade-offs you made (for example, your chosen confidence threshold).
3. Gather all required submission files (see the Result section below) into one folder.
4. Submit your project through the LMS.

Quick-Reference Summary

Use this checklist to confirm you have covered everything before you submit.

#	Checklist item
1	FastAPI microservice built with a working <code>/linkedin</code> endpoint

2	Microservice tested locally with curl and returns ideas, draft, confidence, and hashtags
3	n8n workflow created with a Schedule Trigger
4	Brand Config node set up and passing data to the microservice
5	HTTP Request node correctly calling the microservice
6	Compose Final node assembling the post text and hashtags
7	Approval Gate checking both confidence score and dry-run flag
8	Slack integration sending draft posts for review
9	LinkedIn node configured (live posting optional during development)
10	Logging captures timestamp, draft, confidence, and routing decision
11	Full workflow tested end-to-end, including error handling
12	README and reflection written

Result

By the end of this project, you will deliver a working n8n workflow that automates LinkedIn content generation, a functioning AutoGen-style microservice, a thorough README detailing setup and run instructions, and documentation summarizing your design decisions and testing outcomes.

Submit the following:

- Microservice code (main.py, .env.example)
- Exported n8n workflow JSON
- Screenshots of successful runs (microservice test, each n8n node, and the full workflow)
- A short reflection on design decisions, challenges, and trade-offs